BÖLÜM 2 · ACO → KOD → GA vs ACO → SONUÇLAR → SONRAKİ ADIM

Starship Kalkışında PID Ayarı: Genetik Algoritma ve Karınca Kolonisi

Bölüm 2 · Sırada ne vardı? Aynı problem, ikinci algoritma (ACO) ve adil karşılaştırma · Melih Demir · Optimizasyon Dersi

Bölüm 1’den hatırlatma: GA ile ne yaptık?

1 — Veri

Kaggle: IFT-6/7/8 booster, ilk 150 s

2 — Temizleme

Tekrarlar, km/h→m/s, polinomla yumuşatma, γ, max-Q

3 — Sahte roket

Fizik modeli: itki, yerçekimi, direnç, kütle, motor gecikmesi

4 — Kontrolcü

PID + ileri besleme + kazanç programlama (2 faz)

5 — Senaryolar

9 zor sürüş: ağırlık, direnç, motor gücü, arıza, gürültü

6 — Puan J

Sapma + son hız + max-Q cezası + yakıt + gaz oynaması

7 — Optimizasyon

GA 6 kazancı arıyor; 10 koşu × 64 nesil

Bölüm 1 sonuçları (GA)

Soru	Cevap
GA ne buldu?	10 koşunun hepsi aynı noktaya ulaştı (J ≈ 0,659); rastgele arama 0,709’da kaldı
Ne kadar hızlı?	En iyinin %1 yakınına ~18. nesilde (≈ 519 deneme) ulaştı; kalan nesiller ince ayar
Ne kadar güvenilir?	10/10 koşu başarılı; std $9{,}3 \cdot 10^{-5}$; rastgele aramadan farkı anlamlı (p = 0,0002)
200 yeni senaryoda test	Elle ayara göre J 0,752 → 0,669; gaz oynaması −%67; max-Q ihlali 0/200
Ablasyon: iki faz ne kazandırdı?	Tek faza göre yalnızca %0,2–0,3; asıl işi ileri besleme yapıyor

Bölüm 2’nin sorusu: aynı problemi ikinci bir yöntemle (Karınca Kolonisi) çözersek GA’nın bulduğu çözüm doğrulanır mı, hangisi daha hızlı ve istikrarlı?

01

Karınca Kolonisi Optimizasyonu (ACO)

#src/optimizasyon.py · aco() · karınca, feromon, arşiv, rehber seçimi ve σ

Karınca Kolonisi (ACO): karıncalar en kısa yolu nasıl bulur?

Benzetme: Karıncalar yiyeceğe rastgele yollarla gider ve geçtikleri yere **feromon** (koku) bırakır. Kısa yolda koku hızlı birikir, diğerleri kokuyu takip eder; kötü yolların kokusu uçar (**buharlaşma**).



Benzetmede	Projede
Karınca	Bir aday set: 6 kazanç (Kp1 … Kd2)
Feromon izi	Arşiv: şimdiye kadarki en iyi 30 set, J'ye göre sıralı
Kokuyu takip etmek	Arşivden bir rehber seç, onun yakınında yeni set dene
Buharlaşma	Arşive giremeyen kötü setler silinir (her turda en iyi 30 kalır)

Dorigo (1992); sürekli sayılar için ACO_R: Socha & Dorigo (2008). Tek cümlede ACO: iyilerin etrafında ara → iyiyi arşive koy → kötüyü unut.

ACO_R: feromon = en iyi 30 çözümün arşivi

Neden ACO_R?

- Klasik ACO **yol seçer** (ayrık): şehirden şehre, kenardan kenara.
- Bizim 6 kazancımız ise **sürekli sayılar** (ör. Kp1 = 0,230).
- **ACO_R** (Socha & Dorigo 2008): feromon tablosu yerine **en iyi k çözümün arşivini** tutar.
- Her karınca arşivden bir **rehber** seçer, onun etrafında **çan eğrisiyle** yeni sayı üretir.

Terim	Formül	Anlamı	Değer
k	—	arşiv boyu (hafıza)	30
w_l	$\exp(-(l-1)^2/2q^2k^2)$	sıra ağırlığı	1. sıra en büyük
q	—	en iyilere odak	0,2
p_l	$w_l / \Sigma w$	rehber seçilme şansı	ilk 10 sıra → %89
σ_i	$\xi \cdot \Sigma \lvert s_e - s_l \rvert /(k-1)$	arama genişliği	arşivin yayılımı
ξ	—	σ çarpanı (buharlaşma)	0,85
x_i	$N(s_l , \sigma)$	yeni karıncanın sayısı	her kazanç ayrı

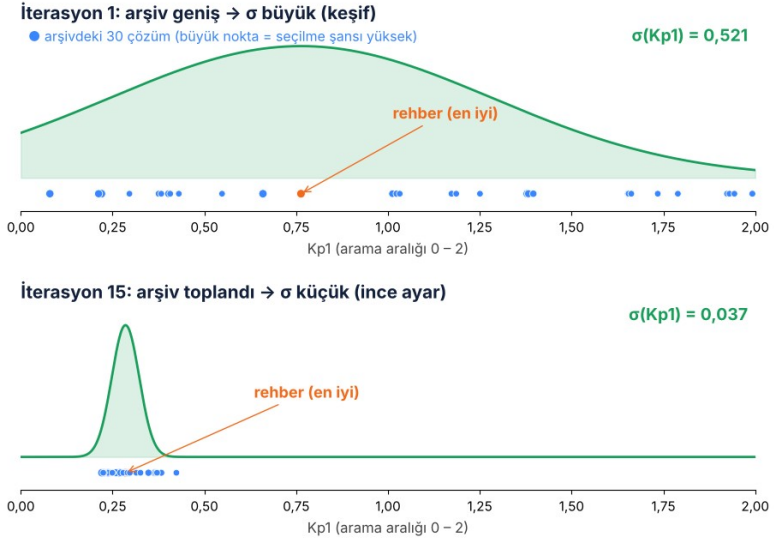
q ve ξ ne yapıyor?

- **q küçük** → hep en iyilerin etrafında ara (sömürü); **q büyük** → arşivdeki herkes rehber olabilir (keşif).
- **ξ büyük** → σ geniş, yavaş daralma (keşif); **ξ küçük** → hızlı daralma, erken takılma riski.

Arşiv toplandıkça σ kendiliğinden küçülür: başta geniş arama, sonda ince ayar. GA'daki sabit büyüklükteki mutasyondan farkı bu.

σ küçülüyor: keşiften ince ayara

İterasyon	En iyi J	Arşivde Kp1 aralığı · σ(Kp1)
1	0,8677	0,08 – 1,99 · σ = 0,521
10	0,6746	0,24 – 0,54 · σ = 0,061
20	0,6611	0,21 – 0,29 · σ = 0,018
30	0,6593	0,221 – 0,245 · σ = 0,005
40	0,6592	0,227 – 0,230 · σ = 0,001
59 (son)	0,6592	0,230 · σ ≈ 0,0001



Koşu 1 (seed 1), Kp1 sütunu. Diğer 5 kazançta da aynı daralma oluyor.

Sağdaki şekil: yeşil çan eğrisi = yeni karıncanın düşebileceği yer.

Kısaca: arşiv toplandıkça arama alanı ~5000 kat daraldı (σ 0,52 → 0,0001); ACO'nun 10 koşusunun neredeyse aynı sayıda bitmesinin sebebi bu.

ACO örnekle: tek düğme (Kp), 4 çözümlük arşiv

Sıra	Ayar (Kp)	Puan (J)
1 (B)	0,9	0,68 ← şans %67
2 (C)	1,4	0,72 ← şans %30
3 (A)	0,2	0,80 ← şans %3
4 (D)	1,8	0,90 ← şans ~%0

- **Dikkat:** seçilme şansı J'nin büyüklüğüne değil **sırasına** bağlı (w_l).
- **Adım 1:** Rastgele 4 Kp seç, uçur, puanla, sırala → arşiv.

- **Adım 2 · Rehber seç:** zar at (%67 · %30 · %3 · ~%0) → B (0,9) çıktı.
- **Adım 3 · σ hesapla:** arşivin B'ye ortalama uzaklığı × ξ = 0,85 × (0,7 + 0,5 + 0,9) / 3 = 0,60.
- **Adım 4 · Örnekle:** çan eğrisinden z = −0,3 geldi → yeni karınca = 0,9 − 0,3 × 0,60 = 0,72.
- **Adım 5 · Puanla, güncelle:** 0,72 → J = 0,66 (yeni en iyi!). 5 çözümden en iyi 4 kalır, D silinir (buharlaşma).
- **Adım 6:** yeni rehber 0,72; σ = 0,85 × (0,18 + 0,68 + 0,52) / 3 = 0,39 → arama daraldı. Tekrarla…

Projede aynısı: 1 yerine 6 sayı, 4 yerine 30 çözümlük arşiv, her turda 30 karınca, 60 iterasyon.

aco() kodu adım adım (1/2): başlangıç arşivi

optimizasyon.py · aco()

```
def aco(f, lb, ub, pop=30, iters=60, seed=0,
        k=30, q=0.2, xi=0.85):
    r = np.random.default_rng(seed)
    lb, ub = np.asarray(lb, float), np.asarray(ub, float)
    D = len(lb)
    A = lb + r.random((k, D)) * (ub - lb)
    FA = f(A)
    s = np.argsort(FA); A, FA = A[s], FA[s]
    rank = np.arange(k)
    w = np.exp(-rank**2 / (2 * (q*k)**2))
    p = w / w.sum()
    egri = [FA[0]]
    n_ev, butce = k, pop * iters
```

Satır	Basitçe
def aco(...)	Girişler: puan makinesi f, sınırlar, 30 karınca, 60 tur; arşiv k = 30, q = 0,2, ξ = 0,85
r = ...(seed)	Rastgele sayı aracı; aynı seed → aynı sonuç
D	6 kazanç
A = lb + ...	Başlangıç arşivi: 30 rastgele set (30 × 6 tablo)
FA = f(A)	30 seti 9 uçuşa sok → 30 J
argsort	Arşivi iyiden kötüye sırala (1. sıra = en küçük J)
rank, w	Sıra ağırlığı: 1. sıra en büyük, sona doğru hızla küçülür
p = w / w.sum()	Seçilme olasılığı (toplam 1); ilk 10 sıra → %89
egri	Her turun en iyi J'si (Şekil 2)
n_ev, butce	Sayaç 30, hak 1800

Buraya kadar: 30 rastgele set → puanla → sırala = feromon arşivi hazır. Seçim şansları bir kez hesaplanıyor.

aco() kodu adım adım (2/2): karıncalar ve arşiv güncelleme

optimizasyon.py · aco() devam

```
    while n_ev < butce:
        m = min(pop, butce - n_ev)
        l = r.choice(k, size=m, p=p)
        sigma = xi * np.abs(A[None, :, :]
                  - A[l][:, None, :]).sum(axis=1) / (k - 1)
        Xn = np.clip(r.normal(A[l], sigma + 1e-12), lb, ub)
        Fn = f(Xn)
        n_ev += m
        A = np.vstack([A, Xn])
        FA = np.concatenate([FA, Fn])
        s = np.argsort(FA)[:k]
        A, FA = A[s], FA[s]
        egri.append(FA[0])
    return A[0], FA[0], np.array(egri)
```

Satır	Basitçe
while …	Ana döngü: 1800 deneme bitene kadar
m	Bu turdaki karınca sayısı: 30
l = r.choice	Her karınca bir rehber seçer (iyi sıra = büyük şans)
sigma	Rehberin her sayısı için: arşivin ona ortalama uzaklığı × ξ
r.normal	Rehber ± σ çan eğrisinden yeni set
np.clip	Sınır kontrolü
Fn = f(Xn)	30 karınca birlikte 9 uçuşa (+30 deneme)
vstack, concatenate	Arşiv + karıncalar = 60 çözüm
argsort[:k]	En iyi 30 kalır, 30'u silinir (buharlaşma)
return	En iyi 6 kazanç, en iyi J, eğri

Ders şeması: f(X) = "sistemi analiz et" · while = geri dönüş oku · butce = "DUR" · arşiv = feromon.

GA ve ACO: aynı problem, farklı mantık

GA (evrim) nasıl arıyor?

- Turnuvayla iki ebeveyn seçer.
- Karıştırır (BLX-α), %15 ihtimalle küçük mutasyon.
- En iyi 2'yi saklar, 28 çocuk üretir.
- Adım büyüklüğü **sabit**: mutasyon hep aralığın ~%10'u.

ACO (karınca) nasıl arıyor?

- Arşivden bir rehber seçer (sıraya göre şans).
- Rehberin etrafında çan eğrisiyle yeni set üretir.
- En iyi 30'u saklar, kalanı siler (buharlaşma).
- Adım büyüklüğü **kendiliğinden küçülür**: σ arşivle daralır.

	GA	ACO
En iyi J / ortalama J	0,65916 / 0,65924	0,65916 / 0,65916
%1'e / %0,1'e ulaşma (ort.)	519 / ~1035 deneme	546 / ~807 deneme
10 koşunun dağılımı (std)	$9{,}3\times10^{-5}$	$2{,}7\times10^{-7}$ (~350 kat dar)

İkisi de aynı kural: alg(f, sınırlar, 30, 60, seed) → (en iyi ayar, en iyi J, eğri) · bütçe 1800 deneme.

ACO ayarları nereden geliyor?

Ayar	Değer	Kaynak / gerekçe
Yöntem	ACO_R (sürekli uzay)	Socha & Dorigo (2008): ACO'nun sürekli sayılar için sürümü
k (arşiv boyu)	30	Önerisi: k, boyut sayısından (6) küçük olmasın; sürüyle eşit seçildi
q (odak)	0,2	Socha & Dorigo çok odaklı $q = 10^{-4}$ kullandı; erken takılmasın diye daha geniş (pratik tercih)
ξ (yayılım)	0,85	Socha & Dorigo (2008) önerdiği değer
Karınca sayısı	30 / tur	GA'daki 30 setle aynı → aynı bütçe
Rehber seçimi	sıraya göre (w_l)	J'nin büyüklüğüne değil sırasına bakar → ölçekten bağımsız
POP × ITER	30 × 60 = 1800	GA ile aynı bütçe: adil karşılaştırma

Hoca sorarsa: "Bu ayarları denediniz mi?" Hayır, duyarlılık analizi yok. Ama ACO'nun 10 bağımsız koşusunun hepsi aynı sonuca ulaştı ($J \approx 0{,}65916$) ve **GA da aynı noktayı buldu** → çözüm iki ayrı yöntemle doğrulandı.

Bildiri "Sınırlılıklar"a: "ACO_R parametreleri (k, q, ξ) literatür ve pratik tercihle seçilmiş, duyarlılık analizi yapılmamıştır."

Koşu › iterasyon › karınca: kaç turda buluyorlar?

Terim	Ne?	Kaç tane?
Karınca	6 sayı: Kp1 Ki1 Kd1 Kp2 Ki2 Kd2	turda 30
İterasyon	30 karınca + arşiv güncelleme	koşuda 60
Koşu	Algoritmanın baştan sona 1 kez çalışması	GA ve ACO 10'ar

İç içe kutular

```
10 koşu  (for s in range(1, 11))
 └─ her koşu = 60 iterasyon  (while)
     └─ her iterasyon = 30 karınca
         └─ her karınca = 6 sayı

GA ve ACO aynı seed'le aynı
30 rastgele setten başlar.
```

Deneme	GA (10 koşu ort. J)	ACO (10 koşu ort. J)
30 (başlangıç)	0,886	0,886
150	0,725 ← GA önde	0,731
300	0,681	0,692
540	0,665 (%1: ~519)	0,665 (%1: ~546)
900	0,660	0,659 ← ACO önde
1800 (son)	0,6592	0,6592

- Başlangıç ikisinde de aynı (aynı seed → aynı 30 rastgele set).
- GA ilk ~500 denemede biraz önde; ~600'den sonra ACO öne geçiyor ve %0,1'e daha erken ulaşıyor (~807'ye karşı ~1035 deneme).

Sonuç: GA kaba aramada biraz hızlı, ACO ince ayarda hızlı ve çok daha istikrarlı. İkisi de aynı J ≈ 0,6592'ye iniyor.

02

GA vs ACO: sonuçlar #src/deneyler.py

Aynı 9 senaryo, ayhı 1800 deneme, 10'ar koşu ve 200 yeni.test senaryosu.

Çıktı: ozet_egitim.csv → Bildiri Tablo 2

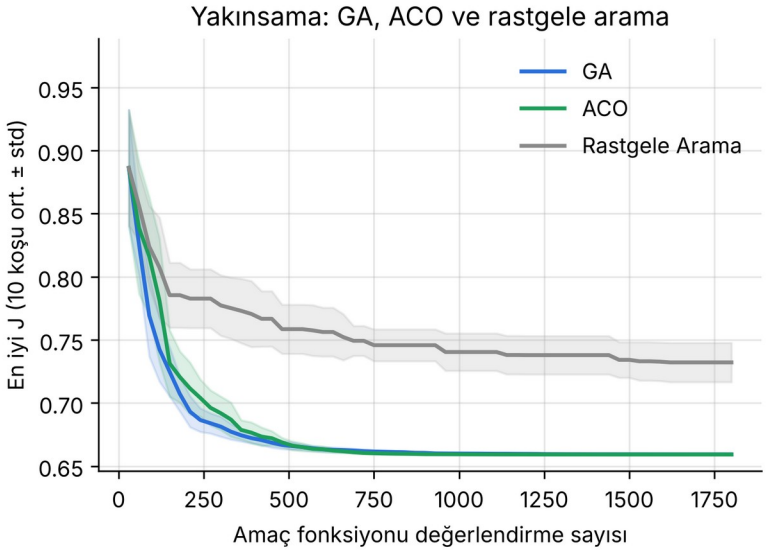
Yöntem	En iyi J	Ortalama J	Std. sapma	%1'e ulaşma	Başarı	Süre/koşu	p (GA ile)
GA	0,65916	0,65924	$9{,}3\times10^{-5}$	519 deneme	10/10	8,8 s	—
ACO	0,65916	0,65916	$2{,}7\times10^{-7}$	546 deneme	10/10	7,7 s	0,0002
Rastgele arama	0,70922	0,73213	$1{,}6\times10^{-2}$	—	0/10	7,9 s	0,0002

Nasıl okunur?

Sütun	Anlamı	Sonuç
En iyi / ortalama J	10 koşunun en iyisi ve ortalaması (küçük = iyi)	İkisi de aynı en iyi noktayı buldu (0,65916)
Std. sapma	10 koşu birbirine ne kadar yakın? (istikrar)	ACO ~350 kat daha dar: her koşu neredeyse aynı sayıyı veriyor
%1'e ulaşma	En iyinin %1 yakınına kaç denemede ulaşıldı (hız)	GA biraz hızlı (519'a karşı 546); fark küçük
p-değeri	Fark şans mı? $p < 0{,}05$ ise gerçek	ACO–GA farkı gerçek ($p = 0{,}0002$) ama J'de yalnızca ~0,0001

Kısaca: iki yöntem aynı çözümü buluyor; ACO daha istikrarlı, GA başta biraz daha hızlı. Rastgele arama yine hiç ulaşamıyor.

Çıktı: Şekil 2 (yakınsama) ve Şekil 3 (kutu grafiği)



Yakınsama: GA (mavi) ilk ~500 denemede biraz önde; ACO (yeşil) ~600'den sonra yetişiyor ve 0,659'a oturuyor. Rastgele (gri) 0,73'te takılıyor.



Kutu grafiği: GA ve ACO kutuları 0,659'da çizgi kadar ince; ACO'nunki daha da ince. Rastgele geniş ve yüksek.

Kaynak: yakinsama.json → Şekil 2 · tum_kosular.csv → Şekil 3 (grafikler.py)

Çıktı: test_sonuclari.csv → Bildiri Tablo 3

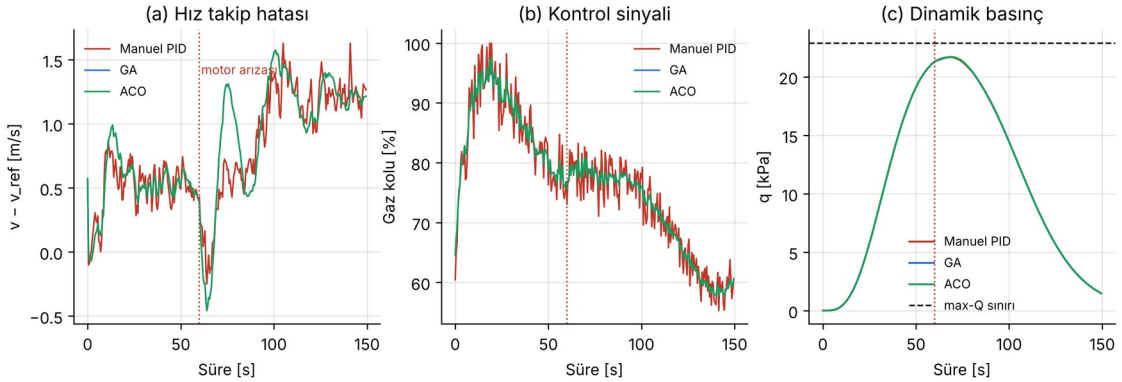
Ayar	J (test)	Sapma IAE	Son hız hatası ort / maks	Max-Q ihlali	En yüksek ivme	Gaz oynaması TV
Manuel PID	0,752	36,2 m	0,19 / 0,73 m/s	0/200	2,39 g	45,6
GA	0,669	38,8 m	0,23 / 0,82 m/s	0/200	2,30 g	15,2
ACO	0,669	38,8 m	0,23 / 0,82 m/s	0/200	2,30 g	15,2
Rastgele arama	0,711	43,2 m	0,17 / 0,72 m/s	0/200	2,35 g	24,4

Nasıl okunur?

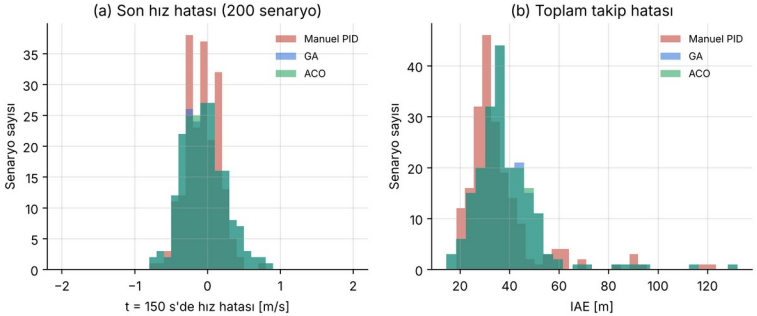
- **J:** GA ve ACO aynı sonucu veriyor: elle ayarlı PID'den %11 iyi (0,752 → 0,669).
- **Kazançlar da aynı:** Kp1 = 0,230 · Ki1 = 0,082 · Kp2 = 0,155 · Ki2 = 0,082 · Kd = 0 (ikisinde de).
- **Gaz oynaması:** %67 azaldı (45,6 → 15,2); max-Q ihlali yok (0/200).
- **Ezber yok:** eğitimde 0,659, testte 0,669 — iki yöntemde de aynı.

Kısaca: iki farklı yöntem 200 yeni senaryoda aynı ayarı doğruladı; bulunan çözüm algoritmaya bağlı değil.

Çıktı: Şekil 4 (motor arızası) ve Şekil 5 (200 test)



- **Şekil 4:** 60. saniyede bir motor bozuluyor (IFT-8).
- (a) Hata kısa süre düşüyor, PID toparlıyor.
- (b) GA (mavi) ve ACO (yeşil) üst üste: aynı kazanç → aynı gaz kolu; manuel (kırmızı) çok daha oynak.
- (c) Basınç sınırın altında.



- **Şekil 5:** 200 testin dağılımı.
- (a) Son hız hatası: hepsi ±1 m/s içinde.
- (b) Toplam sapma (IAE): GA ve ACO histogramları çakışıyor → aynı çözüm.

Kaynak: test_sonuclari.csv'deki ayarlar → grafikler.py yeniden uçurup çiziyor.

03

Özet ve sonraki adım

Ne bulduk ve final bildirisine ne giriyor?

Genel özet

1. **Veri**
   Kaggle: IFT-6/7/8 booster, ilk 150 s
2. **Temizleme**
   Tekrarlar, km/h→m/s, polinomla yumuşatma, γ, max-Q
3. **Sahte roket**
   Fizik modeli: itki, yerçekimi, direnç, kütle, motor gecikmesi
4. **Kontrolcü**
   PID + ileri besleme + kazanç programlama (2 faz)
5. **Senaryolar**
   9 zor sürüş: ağırlık, direnç, motor gücü, arıza, gürültü
6. **Puan J**
   Sapma + son hız + max-Q cezası + yakıt + gaz oynaması
7. **Optimizasyon**
   GA ve ACO 6 kazancı arıyor; 10'ar koşu × 1800 deneme

Sonuçlar

Soru	Cevap
Hangisi daha iyi çözümü buldu?	İkisi de aynı noktaya ulaştı (J ≈ 0,6592); kazançlar neredeyse aynı
Hangisi daha hızlı?	%1'e GA biraz önce (519 / 546); %0,1'e ACO önce (~807 / ~1035)
Hangisi daha istikrarlı?	ACO: std $2{,}7 \cdot 10^{-7}$ (GA'dan ~350 kat dar); fark anlamlı (p = 0,0002)
200 yeni senaryoda test	İkisi de: J 0,752 → 0,669; gaz oynaması −%67; max-Q ihlali 0/200
Rastgele arama	0,709'da kaldı → iki algoritma da gerçekten öğreniyor

Kısaca: iki farklı yöntem aynı ayarı buldu → çözüm güvenilir. GA kaba aramada biraz hızlı, ACO ince ayarda hızlı ve çok daha istikrarlı.

SONRAKİ ADIM

Final bildirisine doğru

1 Bildiriyi tamamlamak

- GA + ACO sonuçları Tablo 2–3 ve Şekil 2–5'e
- Sınırlılıklar: algoritma ayarları ve J ağırlıkları tasarımcı seçimi
- Son teslim: 4 Aralık 2026

2 Daha gerçekçi model

- İki boyutlu uçuş: yatay hareket ve eğim
- Diğer uçuşların verisi (IFT-1...5)
- Farklı arıza ve gürültü senaryoları

3 Yöntemi geliştirmek

- Duyarlılık analizi: q, ξ ve GA ayarları
- Hibrit GA–ACO (ödevde ek puan)
- Kodları GitHub'da paylaşmak (ek puan)

Hedef: İrem'in iniş çalışmasıyla birlikte, Booster'ın kalkışından yakalamaya kadar tüm uçuşunu kapsayan ortak bildiri.